



Universidad Nacional Autónoma de México
Facultad de Ingeniería
División de Ingeniería Eléctrica



Carrera: Ingeniería en Computación

Profesor: Óscar Manuel Ruíz Hurtado

Curso: Sistemas Distribuidos

S07 - Bitácora de MexiBanco Distribuido

Alumno: Rodríguez Zamora Joshua

Fecha de entrega: 03/10/2026

Semestre: 2027-1

Grupo: 02

Contenido

1. La historia:	3
2. El directorio:	3
3. Gateway:.....	7
3.1. Rutas de Gateway:.....	7
4. Copias de cuenta:.....	9
4.1. Mira como reparte:.....	10
5. ¿Y si una copia muere?.....	11
6. Cierre:.....	13
TAREA:.....	14

1. La historia:

Antes, en el monolito la transferencia se manejaba en una sola aplicación con una sola base de datos. Se utilizaban métodos desde transferenciaService hasta cuentaService. Si alguna transferencia fallaba, la base también se deshacía del cargo. Sin embargo, al distribuir en 5 servicios cada programa está separado en su propio contenedor y con su propia base de datos. Ahora se hacen peticiones por HTTP y cada transferencia puede presentar problemas, pues, en caso de que la red falle o se caigan los servicios, puede haber un cargo que se realizó a mitad del fallo, haciendo que la transferencia nunca se completara, es decir, que el dinero no llegó a la cuenta destino y dicho cargo de la cuenta origen no se puede deshacer. Ante esto, aprenderemos a manejar los servicios, creando un directorio que nos permite visualizar el estado de los contenedores, establecer por donde pueden provenir y dirigirse las peticiones y finalmente repartir el tráfico entre copias.

2. El directorio:

Estado de los contenedores: (Nota: Hice la captura al final de la práctica)

Realizado 15:08 pm



```
jack19@amalgatic-intergalactic: MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps | grep -vE
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
mexi-banco-v06-db-1                 postgres:17-alpine                 "docker-entrypoint.s..."           db         2 hours ago    Up 2 hours    (healthy)    5432/tcp
mexi-banco-v06-movimiento-1         sha256:b0de023e8085b6e41b5c7442baff73933245f90e083af86f721f7efba8716c0a    "java -jar app.jar"                 movimiento 2 hours ago    Up 2 hours    (healthy)    0.0.0.0:8082->8080/tcp
mexi-banco-v06-notificacion-1       sha256:fcd57049f88a46937189b0b07e39b569107fd1869536f5fce71f712f75ac9c9b    "java -jar app.jar"                 notificacion 2 hours ago    Up 2 hours    (healthy)    0.0.0.0:8084->8080/tcp
mexi-banco-v06-spei-1               sha256:f7aa30bbf0fe07f0bef8bb18cdec7abfae599306c3fae6fb8cb17d5f701ec5b3    "java -jar app.jar"                 spei        2 hours ago    Up 2 hours    (healthy)    0.0.0.0:8085->8080/tcp
mexi-banco-v06-transferencia-1      sha256:7e8b92f3d38773f998a89e5b3323cc7e1ab4f8d83a682918f2c5574e2acabe98    "java -jar app.jar"                 transferencia 2 hours ago    Up 2 hours    (healthy)    0.0.0.0:8083->8080/tcp
```

imagen 1. Contenedores y puertos asignados.

Buscamos el puerto de cuenta sólo para encontrar que no hay cuentas con clabe 000, un error 404.

Realizado después de terminar para obtener la evidencia.

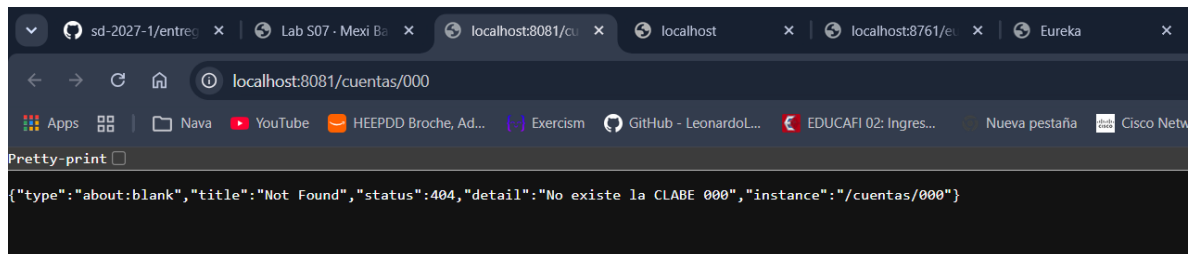


imagen 2. Puerto 8081. Cuenta.

El directorio se maneja con Eureka, al inicio de la práctica este no esta activado, ni asignado al puerto 8671 y no podemos acceder a él como se muestra a continuación:

Realizado 15:09 pm

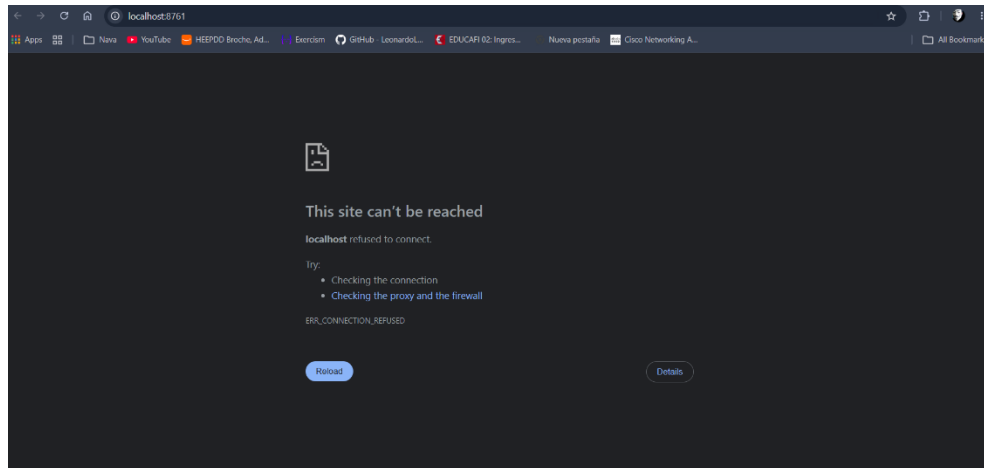


imagen 3. Puerto 8671 inaccesible.

Después, se añadieron las sentencias necesarias en cada servicio en su respectivo pom.xml para añadirlos al directorio general para que ahora sí, entrando al puerto local 8671 se pudiera mostrar la siguiente página con todos los servicios arriba:

Realizado 15:10 pm

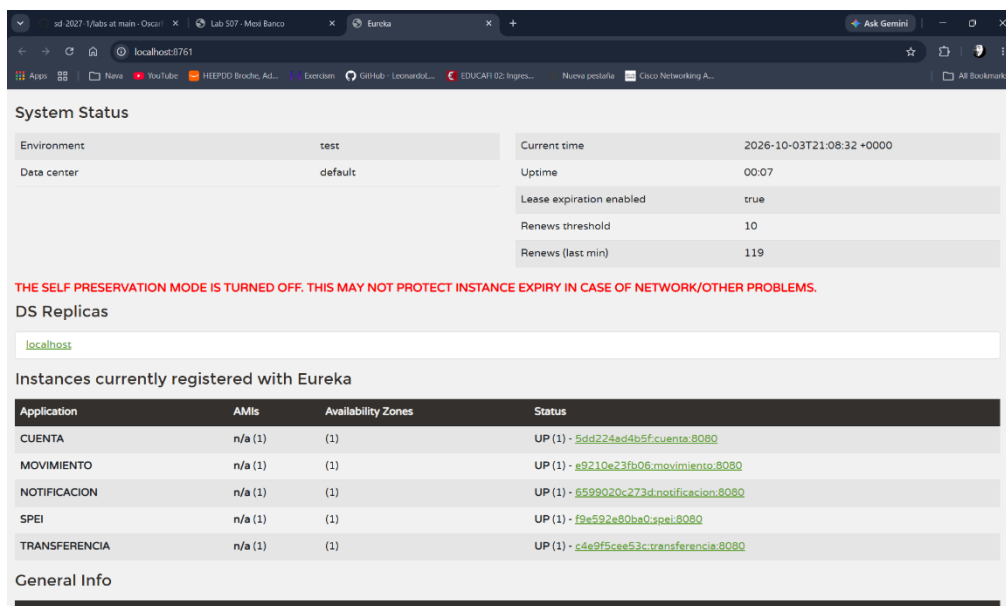


imagen 4. Directorio arriba y funcional.

Cuenta XML: En esta parte se muestra la parte del XML donde podemos ver algunas configuraciones del contenedor de cuenta, esto es lo que el directorio conoce sobre cuenta, como su hostname, su puerto, el estado y el tiempo de heartbeat y plazo del compose.

Realizado 15:11 pm

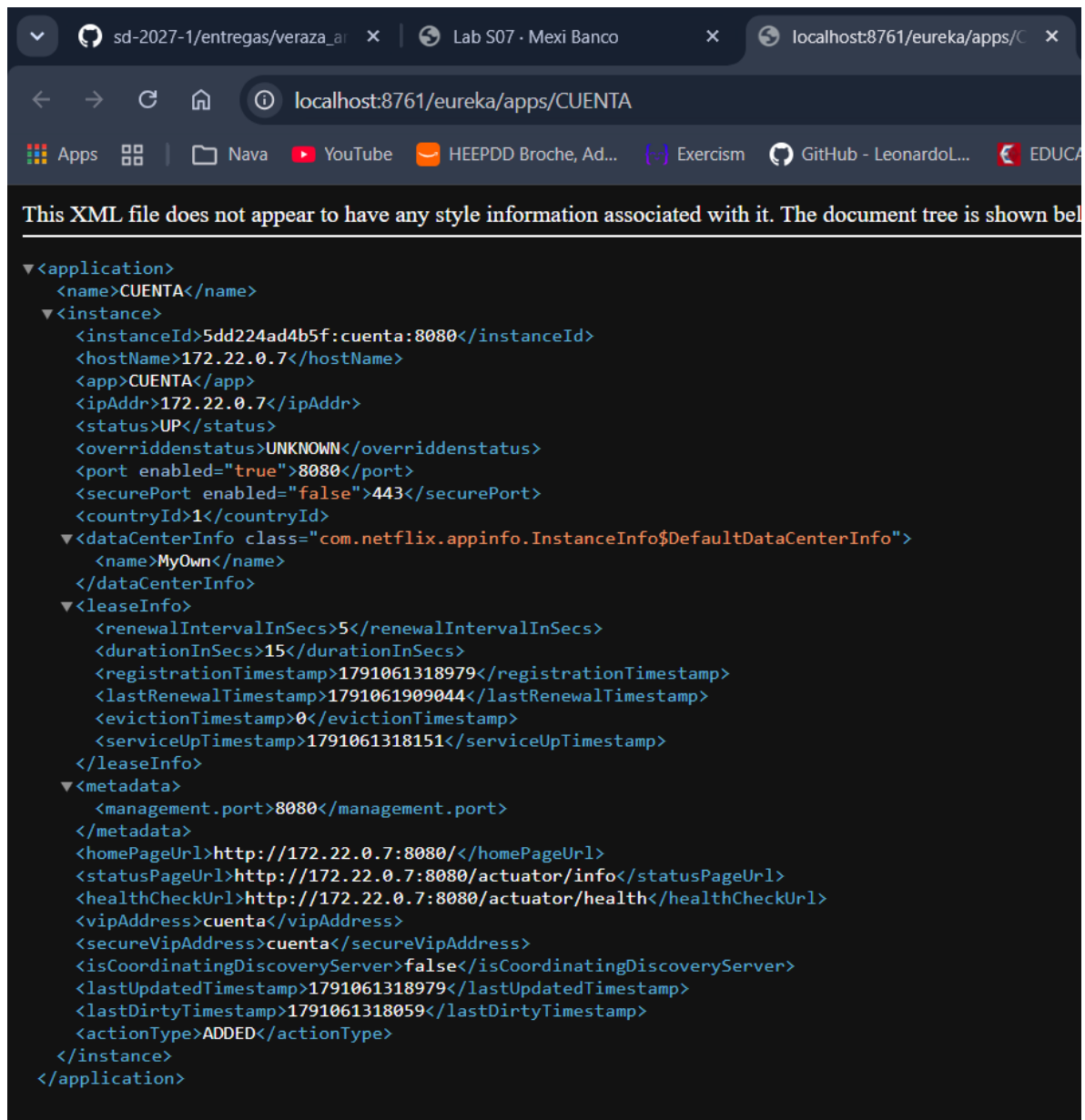


imagen 5. XML de cuenta.

Experimento opcional: Realizado a las 15:13 pm

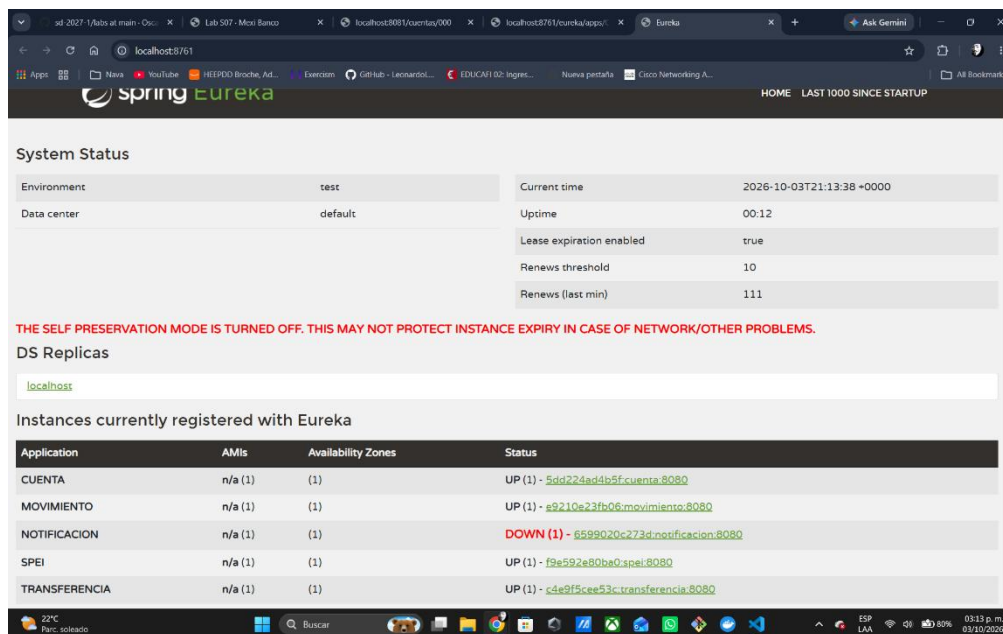


imagen 6. Apagar notificación.

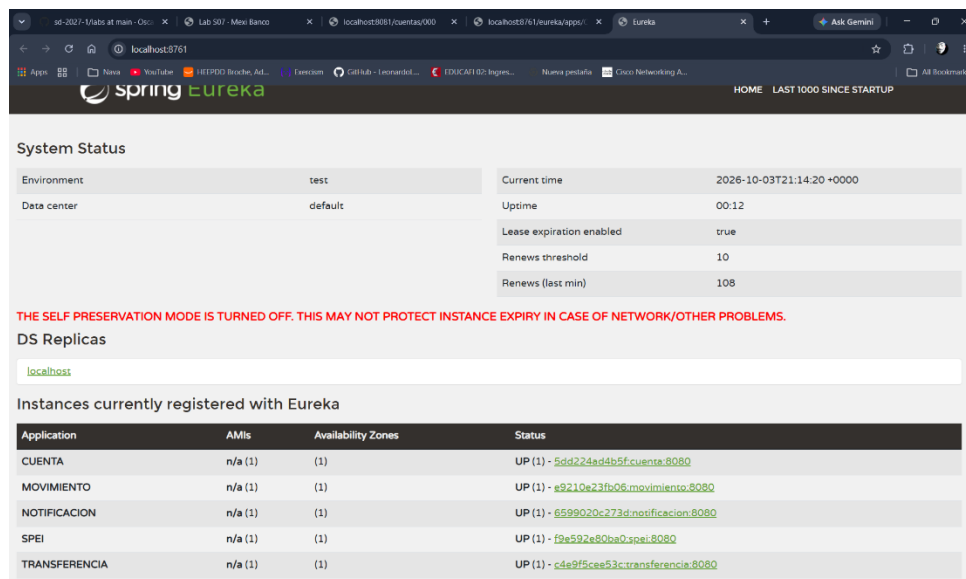


imagen 7. Reiniciar notificación.

PIENSA: No, se sigue usando la dirección <http://cuenta:8080>, esta dirección se encuentra en el Docker-compose.yml. Se utiliza debido a la red interna entre contenedores de Docker, por lo tanto no se utiliza el directorio para las comunicaciones

3. Gateway:

Antes de colocar el Gateway se hace un POST a movimientos, pero faltaron datos y se devuelve un error 400, indicando que el endpoint existe pero que la petición no fue completa.

Realizado 15:17 pm.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 03 Oct 2026 21:16:50 GMT
Connection: close
{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

imagen 8. Error 400.

3.1. Rutas de Gateway:

Se levanta el Gateway añadiéndole las rutas a cada servicio, así como las operaciones permitidas en cada uno, después se prueba:

Después de configurar y resolver pendientes en casa, se continúa a las 15:36 pm.

En este caso de prueba el Gateway funcionó, se encontró la ruta de salida, pero se le preguntó a cuentas por la cuenta 000 y está no existe.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 5dd224ad4b5f
Content-Type: application/problem+json
Date: Sat, 03 Oct 2026 21:36:13 GMT
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
```

imagen 9. Petición de una cuenta no existente.

Aquí hubo error 404 porque no se soportan operaciones POST en movimientos por ningún Gateway.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130
{"timestamp":"2026-10-03T21:36:23.426+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"032b074a-99"}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$
```

imagen 10. Error 404, no se soportan los POST.

Creación de cuenta: S071791072442, Se realizó de nuevo una nueva cuenta porque olvidé tomar captura como evidencia. (hora 18:11 pm en el desarrollo del reporte).

Se crea y arroja el código 201.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 31d6a8aa6960
Content-Type: application/json
Date: Sun, 04 Oct 2026 00:07:22 GMT

{"id":2,"clabe":"S071791072442","titular":"S07","saldo":0}
```

imagen 11. Nueva cuenta generada. Código 201.

Después se generó una consulta que arrojó el código 200 y se muestra el saldo en 0.00.

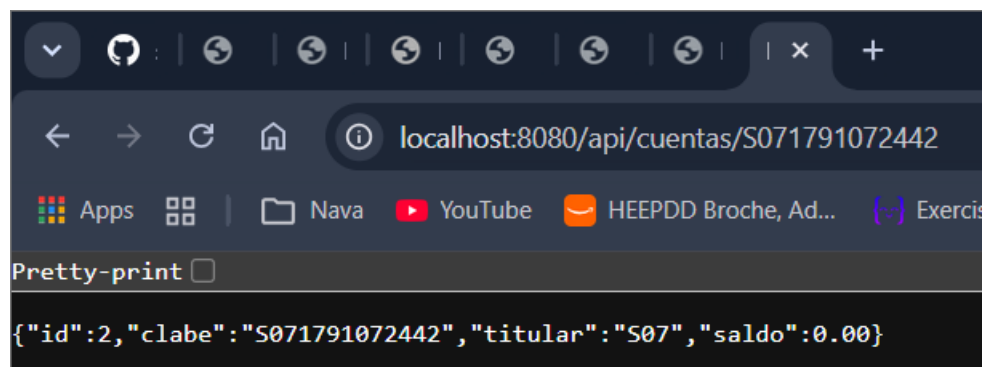
```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 31d6a8aa6960
Content-Type: application/json
Date: Sun, 04 Oct 2026 00:09:24 GMT

{"id":2,"clabe":"S071791072442","titular":"S07","saldo":0.00}
```

imagen 12. Consulta, código 200.

Finalmente se muestra en el navegador la cuenta generada por el Gateway.



```
localhost:8080/api/cuentas/S071791072442

Pretty-print ☐

{"id":2,"clabe":"S071791072442","titular":"S07","saldo":0.00}
```

imagen 13. Cuenta generada por el gateway.

4. Copias de cuenta:

Se crean copias que se muestran en el directorio, dirigidas para el servicio de cuenta, con lo que buscamos simular la llegada y repartición de peticiones después.

Realizado 15:44 pm

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
39e91a22838f mexi-banco-v06-cuenta-2 8080/tcp
e58af0e2d963 mexi-banco-v06-cuenta-1 8080/tcp
8e996a1a537a mexi-banco-v06-cuenta-3 8080/tcp
```

imagen 14. Generación de 3 copias.

Estas copias se muestran en el directorio por medio del status de cuenta, donde se muestra UP (3) seguido de los identificadores de los 3 contenedores dedicados a cuenta.

The screenshot shows the Eureka web interface in a browser. At the top, there's a configuration table:

Lease expiration enabled	true
Renews threshold	0
Renews (last min)	144

Below this, a red warning message states: "THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS."

The "DS Replicas" section shows "localhost".

The "Instances currently registered with Eureka" section contains a table:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 39e91a22838f:cuenta:8080, e58af0e2d963:cuenta:8080, 8e996a1a537a:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - b6cac13e3c1f:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - e9210e23fb06:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 6599020c273d:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - f9e592e80ba0:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - c4e9f5cee53c:transferencia:8080

The "General Info" section shows:

Name	Value
total-avail-memory	99mb
num-of-cpus	16

imagen 15. Copias generadas.

4.1. Mira como reparte:

En esta sección se simulan 9 consultas hacia la cuenta recién creada, y como se dividen en cada una por medio de round robin, yendo 1 a 1 en diferentes contenedores como se muestra a continuación. **(Realizado 15:46 pm):**

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 39e91a22838f
X-Instancia: e58af0e2d963
X-Instancia: 8e996a1a537a
X-Instancia: 39e91a22838f
X-Instancia: e58af0e2d963
X-Instancia: 8e996a1a537a
X-Instancia: 39e91a22838f
X-Instancia: e58af0e2d963
X-Instancia: 8e996a1a537a
```

imagen 16. Generación de consultas.

Finalmente se muestran las consultas, mostrando que hay 3 en cada copia.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose logs -f cuenta | grep atendida
cuenta-2 | 2026-10-03T21:44:47.094Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 39e91a22838f
cuenta-2 | 2026-10-03T21:44:48.292Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 39e91a22838f
cuenta-2 | 2026-10-03T21:44:48.721Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 39e91a22838f
cuenta-1 | 2026-10-03T21:44:47.505Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por e58af0e2d963
cuenta-1 | 2026-10-03T21:44:48.451Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por e58af0e2d963
cuenta-1 | 2026-10-03T21:44:48.942Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por e58af0e2d963
cuenta-3 | 2026-10-03T21:44:47.893Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 8e996a1a537a
cuenta-3 | 2026-10-03T21:44:48.587Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 8e996a1a537a
cuenta-3 | 2026-10-03T21:44:49.150Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791063505 atendida por 8e996a1a537a
```

imagen 17. Consultas y repartición.

Piensa, Esto prueba que el Gateway reparte. ¿Y cuando transferencia llama a cuenta por dentro, también reparte? Mira otra vez su CUENTA_URL en el compose:

No, como transferencia tiene la URL de cuenta con su puerto, es decir, <http://cuenta:8080>, transferencia no hace uso del directorio y sólo usa la red interna de Docker, usando la url, sin saber cuantas copias hay, además de que todas estas copias utilizan “el mismo puerto” por lo que el DNS de Docker es quien busca la dirección y manda las peticiones por un solo puerto.

5. ¿Y si una copia muere?

Predicción. ¿Van a fallar todas las consultas, ninguna o algunas? ¿Durante cuánto tiempo? ¿Dejan de fallar en cuanto la copia desaparece del directorio, o antes, o después?

Yo considero que fallará una sola consulta, ya que no se repite el comando constantemente y supongo que sólo matará a la primera copia que encuentre.

Además, fallará permanentemente hasta que sea restablecida, ya que en la práctica se menciona que se dejarán de mandar heartbeats.

Y, por último, a mi parecer no siento que se detengan las fallas después de desaparecer del directorio, pienso que se mantendrá caído hasta que alguien levante la copia.

Utilizando dos terminales, utilizamos el comando Kill (15:51 pm):

```
jackl@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
15:50:18 200
15:50:19 200
15:50:20 200
15:50:21 200
15:50:22 500
15:50:23 200
15:50:24 200
15:50:25 200
15:50:26 200
15:50:27 200
15:50:28 500
15:50:29 200
15:50:30 200
15:50:31 500
15:50:32 200
15:50:33 500
15:50:34 200
15:50:35 200
15:50:36 200
15:50:37 200
15:50:38 200
15:50:39 500
15:50:40 500
15:50:41 500
15:50:42 500
15:50:43 500
15:50:44 200
15:50:45 200
15:50:46 200
15:50:47 200
15:50:48 200
15:50:49 200
15:50:50 500
15:50:51 200
15:50:52 200
15:50:53 200
15:50:54 200
15:50:55 200
15:50:56 200
15:50:57 200
15:50:58 200
15:50:59 200
15:51:00 200
15:51:01 200
15:51:02 200
15:51:03 200
15:51:04 200
15:51:05 200
15:51:06 200
15:51:07 200
15:51:08 200
15:51:09 200
15:51:10 200
15:51:11 200
15:51:12 200
15:51:13 200
15:51:14 200
15:51:15 200
15:51:16 200
15:51:17 200
15:51:18 200
15:51:19 200
15:51:20 200
15:51:21 200
15:51:22 200
15:51:23 200
15:51:24 200
15:51:25 200
15:51:26 200
15:51:27 200
15:51:28 200
15:51:29 200
15:51:30 200
15:51:31 200
15:51:32 200
15:51:33 200
15:51:34 200
15:51:35 200
```

imagen 18. Consulta por segundo en primera terminal

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ date +%T # anota la hora del apagado
15:50:19
docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ |
```

imagen 19. Kill, en segunda terminal.

La copia de mexi-banco-v06-cuenta1 cayó aproximadamente a las **15:50:55 pm**

System Status

Environment	test	Current time	2026-10-03T21:50:55 +0000
Data center	default	Uptime	00:49
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	184

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - 39e91a22838f:cuenta:8080 , 8e996a1a537a:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - b6cac13e3c1f:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - g9210e23fb06:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 6599020c273d:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - f9e592e80ba0:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - c4e9f5cee53c:transferencia:8080

General Info

imagen 20. Cuenta UP(2).

En mi caso salieron 6 errores, todos con el código 500, mientras que la copia salió del directorio aproximadamente 37 segundos después. Esto ocurre porque fue una caída inesperada, falló y nadie avisó al respecto, por ello Docker esperó 15 segundos de plazo para recibir un heartbeat, además de revisiones periódicas cada 5 segundos, a la vez que el Gateway sigue mandando peticiones a la copia muerta. Que no responde.

Ahora revivimos a la copia 1: **15:53 pm**

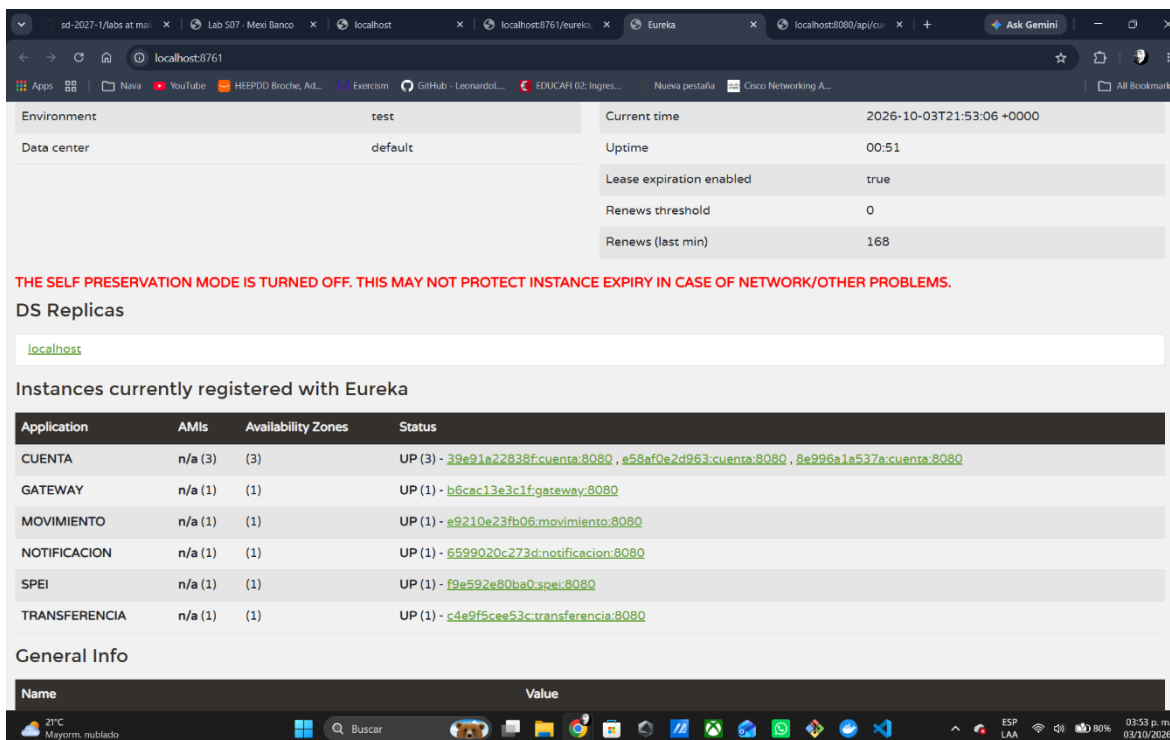


imagen 21. UP (3) en Cuenta.

6. Cierre:

- **El directorio resuelve:** la visualización simplificada de los servicios interconectados que tenemos e incluso podemos ver las copias creadas. Permite ver sus datos y los puertos que utilizan y con los que se intercomunican.
- **El gateway resuelve:** la comunicación con los demás servicios y con el exterior.
- **El balanceador resuelve:** la repartición de carga o peticiones por turnos por medio de round robin, entre las copias disponibles y deja de enviar peticiones a las que fallan.
- **Cuando murió una copia:** vi errores intermitentes por el periodo en que el sistema se enteraba de la caída de la copia porque el sistema cree que la copia sigue activa por un rato, mientras espera el heartbeat y sigue mandando tráfico hasta que se reconoce que ha caído la copia.

TAREA: Realizado a las 19:34 pm junto al reporte

```
jackl1@Mamalactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ ./demo-v06.sh --latido
```

== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ==

-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):

```
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
```

-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ==

```
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009107751801","titular":"Ana","saldoInicial":1000}'
{"id":5,"clabe":"002180009107751801","titular":"Ana","saldo":1000}
[HTTP 201]
```

-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

== 2. Abrir la cuenta de Beto con 500 (misma puerta) ==

```
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009107751802","titular":"Beto","saldoInicial":500}'
{"id":6,"clabe":"002180009107751802","titular":"Beto","saldo":500}
[HTTP 201]
```

== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==

```
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009107751801","claveDestino":"002180009107751802","monto":200}'
{"id":2,"claveOrigen":"002180009107751801","claveDestino":"002180009107751802","monto":200,"fecha":"2026-10-04T01:32:01.552354097Z"}
[HTTP 201]
```

-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:

```
cuenta-3 GET /cuentas/002180009107751801 atendida por 26e13751e921
cuenta-1 GET /cuentas/002180009107751802 atendida por 53102ba73185
cuenta-2 POST /cuentas/002180009107751801/cargos atendida por f13995f2772a
cuenta-3 POST /cuentas/002180009107751802/abonos atendida por 26e13751e921
```

-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

== 4. Consultar esa transferencia ==

```
$ curl http://localhost:8080/api/transferencias/2
{"id":2,"claveOrigen":"002180009107751801","claveDestino":"002180009107751802","monto":200.00,"fecha":"2026-10-04T01:32:01.552354Z"}
[HTTP 200]
```

== 5. Saldos: el dinero no se crea ni se pierde ==

```
$ curl http://localhost:8080/api/cuentas/002180009107751801
{"id":5,"clabe":"002180009107751801","titular":"Ana","saldo":800.00}
[HTTP 200]
```

```
$ curl http://localhost:8080/api/cuentas/002180009107751802
{"id":6,"clabe":"002180009107751802","titular":"Beto","saldo":700.00}
[HTTP 200]
```

-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ==

```
$ curl http://localhost:8080/api/movimientos?clabe=002180009107751801
{"id":8,"clabeCuenta":"002180009107751801","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-04T01:32:01.536997Z","referencia":"a 002180009107751802"},{"id":6,"clabeCuenta":"002180009107751801","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-04T01:32:00.017532Z","referencia":"saldo inicial"}
[HTTP 200]
```

```
$ curl http://localhost:8080/api/notificaciones?clabe=002180009107751802
{"id":5,"clabeCuenta":"002180009107751802","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-04T01:32:01.577030Z"}
[HTTP 200]
```

== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ==

```
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91077518' -d '{"claveOrigen":"002180009107751801","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50}'
{"id":3,"idempotencyKey":"spei-demo-91077518","claveOrigen":"002180009107751801","bancoDestino":"BANCO-X","claveDestino":"01218000000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
```

```
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91077518' -d '{"claveOrigen":"002180009107751801","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50}'
{"id":3,"idempotencyKey":"spei-demo-91077518","claveOrigen":"002180009107751801","bancoDestino":"BANCO-X","claveDestino":"01218000000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
```

-> mismo id (3 -> 3): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ==

```
consulta 1 X-Instancia: 26e13751e921 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: 53102ba73185 (mexi-banco-v06-cuenta-1)
consulta 3 X-Instancia: f13995f2772a (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: 26e13751e921 (mexi-banco-v06-cuenta-3)
consulta 5 X-Instancia: 53102ba73185 (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: f13995f2772a (mexi-banco-v06-cuenta-2)
consulta 7 X-Instancia: 26e13751e921 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 53102ba73185 (mexi-banco-v06-cuenta-1)
consulta 9 X-Instancia: f13995f2772a (mexi-banco-v06-cuenta-2)
```

-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas

-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

== 9. Los endpoints internos no tienen puerta ==

```
$ curl -X POST http://localhost:8080/api/cuentas/002180009107751801/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-04T01:32:11.086+00:00","path":"/api/cuentas/002180009107751801/abonos","status":404,"error":"Not Found","requestId":"057b34b1-3027"}
[HTTP 404]
```

-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

== 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ==

```
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009107751801","claveDestino":"002180009107751899","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009107751899","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
```

```
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009107751801","claveDestino":"002180009107751802","monto":99999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009107751801","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]
```

== 11. Los saldos no se movieron con los errores ==

-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

```

== 12. El latido: matar una replica de cuenta sin que se despida ==
-> en el directorio hay 3 instancias de CUENTA; la victima es 53102ba73185 (mexi-banco-v06-cuenta-1)
-> docker kill = se va la luz: el proceso no alcanza a darse de baja. Solo dejan de llegar sus latidos.
-> (docker stop seria un apagado ordenado: Spring la marca DOWN al instante y nadie le manda trafico)
t= 0s CUENTA en el directorio: 3 por la puerta: 1 bien, 0 con error
t= 25s CUENTA en el directorio: 3 por la puerta: 9 bien, 4 con error
t= 30s CUENTA en el directorio: 3 por la puerta: 10 bien, 5 con error
t= 35s CUENTA en el directorio: 2 por la puerta: 11 bien, 6 con error
-> el directorio saca a la replica muerta a los 35 s
-> mientras tanto, por la puerta: 11 respuestas bien y 6 con error (el ultimo error a los 31 s)
-> los errores son el gateway mandando peticiones a una replica que ya no existe: el registro es eventualmente consistente
Lo que sabe ahora el directorio:
CUENTA 2 instancia(s)
GATEWAY 1 instancia(s)
MOVIMIENTO 1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI 1 instancia(s)
TRANSFERENCIA 1 instancia(s)

== 13. Volver a prender la replica y ver cuanto tarda en reaparecer ==
-> mexi-banco-v06-cuenta-1 aparece otra vez en el directorio a los 11 s de docker start (arranque de la JVM + registro)
-> y el gateway le vuelve a mandar peticiones a los 18 s

Fin. Con EUREKA_EXPIRA_S=15 EUREKA_CONSULTA_S=5 EUREKA_LATIDO_S=5 BALANCEADOR_CACHE=5s (Spring Cloud trae latido 30, expiracion 90, consulta 30 y cache 35s).

```

imagen 23. Prueba de simulación.

```

jack10@Mamallactic-intergalactic MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-1 | 2026-10-04T01:30:52.577Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107744701/cargos atendida por 53102ba73185
cuenta-3 | 2026-10-04T01:30:52.717Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107744702/abonos atendida por 26e13751e921
cuenta-1 | 2026-10-04T01:31:07.863Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107744701/cargos atendida por 53102ba73185
cuenta-1 | 2026-10-04T01:32:04.743Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107751801/cargos atendida por 53102ba73185
cuenta-2 | 2026-10-04T01:31:00.621Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107744701/cargos atendida por f13995f2772a
cuenta-3 | 2026-10-04T01:32:01.521Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107751802/abonos atendida por 26e13751e921
cuenta-3 | 2026-10-04T01:32:11.516Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107751801/cargos atendida por 26e13751e921
cuenta-2 | 2026-10-04T01:32:01.504Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009107751801/cargos atendida por f13995f2772a

```

imagen 22. Logs de cargos y abonos.

Se obtuvieron los resultados esperados, 800 pesos en la cuenta de Ana y 700 en la de Beto después de la transferencia y tiempo después 750 en la cuenta de Ana y los mismos 700 en la cuenta de Beto tras el SPEI.

Y los saldos no se movieron a pesar de los errores.

Mientras que al buscar los latidos cuando se muere una replica se muestra el tiempo de reconocimiento que tarda el sistema en darse cuenta de que se ha caído una copia. Tomando 35 segundos en retirar la copia muerta, mientras que al revivir a la replica se tardó 11 segundos para reaparecer en el directorio.